UNIVERSITÀ DI PISA

Sorting Cell

De Carlo Samuele
Chelini Matteo

Progettazione di Sistemi Microelettronici
Modulo digitale

Anno Accademico 2024/2025

Indice

1 Introduzione **3**
1.1 Sorting Cell . . . 3
1.2 Algoritmi di ordinamento . . . 3
1.3 Requisiti e vincoli di progetto . . . 3

2 Principio di funzionamento **4**
2.1 SIPO . . . 4
2.1.1 Vantaggi del SIPO . . . 5
2.1.2 Codice VHDL del blocco SIPO . . . 5
2.2 Sorter . . . 7
2.2.1 Odd-Even Transposition Sort . . . 7
2.2.2 Latenza . . . 7
2.2.3 Uscita e segnalazione . . . 7
2.2.4 Codice VHDL del blocco Sorter . . . 8
2.3 PISO . . . 10
2.3.1 Codice VHDL del blocco PISO . . . 11

3 Strategia di test **11**
3.1 Primo test: normale funzionamento . . . 11
3.2 Secondo test: block overflow . . . 11
3.3 Terzo test: inserimento di meno di M simboli . . . 12
3.4 Quarto test: conflitto lettura/scrittura . . . 12
3.5 Quinto test: lettura in assenza di dati . . . 12
3.6 Sesto test: altri possibili scenari . . . 12

4 Sintesi e implementazione su Vivado **13**
4.1 Wrapper . . . 13
4.2 Sintesi logica . . . 13
4.3 Implementazione . . . 14
4.3.1 Risorse utilizzate . . . 15
4.3.2 Consumo di potenza . . . 15
4.3.3 Warning . . . 16

5 Conclusioni **16**

1 Introduzione

Il progetto assegnato riguarda la descrizione **VHDL** di un **Sorting Cell**. Lo scopo del documento è quello di descriverne il funzionamento, le scelte implementative, la gestione degli errori, la strategia di test per verificarne la correttezza, l'analisi della sintesi e l'implementazione su FPGA.

1.1 Sorting Cell

Il **Sorting Cell** è un blocco fondamentale nei circuiti progettati per **l'ordinamento hardware**. Esso riceve in ingresso un insieme di valori e li restituisce in uscita ordinati secondo un criterio prestabilito, ad esempio crescente o decrescente. Grazie alla sua natura **parallela**, il blocco è in grado di eseguire l'ordinamento molto più velocemente rispetto a un algoritmo software tradizionale.
L'impiego di un Sorting Cell come **acceleratore hardware** è particolarmente utile in applicazioni ad alte prestazioni, come il **machine learning**, **l'elaborazione di segnali** o **sistemi embedded** che devono gestire grandi quantità di dati in tempo reale. Inoltre, questi blocchi sono **modulari** e **scalabili**, rendendo semplice l'implementazione di circuiti capaci di ordinare array di dimensioni maggiori senza modificare la logica di base.

1.2 Algoritmi di ordinamento

Nei circuiti di ordinamento, diversi algoritmi possono essere implementati per organizzare i dati in modo efficiente. Tra i più comuni ci sono il **Bitonic Sort**, il **Merge Sort** e **l'Odd-Even Sort**, ciascuno con caratteristiche diverse in termini di parallelismo e complessità. L'Odd-Even Sort, in particolare, sfrutta confronti e scambi tra elementi adiacenti in modo alternato, permettendo di ordinare sequenze in modo semplice ma efficace su hardware parallelo. La scelta dell'algoritmo dipende dalle esigenze specifiche dell'applicazione, come la velocità, la scalabilità e la facilità di implementazione su FPGA o altri acceleratori hardware.
Dato che per il progetto era richiesto un **algoritmo semplice e facilmente implementabile**, è stato scelto l'algoritmo **Odd-Even**, che risulta comunque efficace per ordinare insiemi di dati di piccola scala (ad esempio decine di elementi). Va però sottolineato che, in termini di prestazioni, esistono algoritmi più veloci e ottimizzati, con maggiore efficienza in termini di tempo di elaborazione e utilizzo delle risorse. Questi **approcci alternativi** verranno discussi successivamente nella sezione dedicata alle possibili implementazioni e varianti.

1.3 Requisiti e vincoli di progetto

Il progetto prevede la realizzazione di un circuito caratterizzato dall'interfaccia riportata in figura 1:

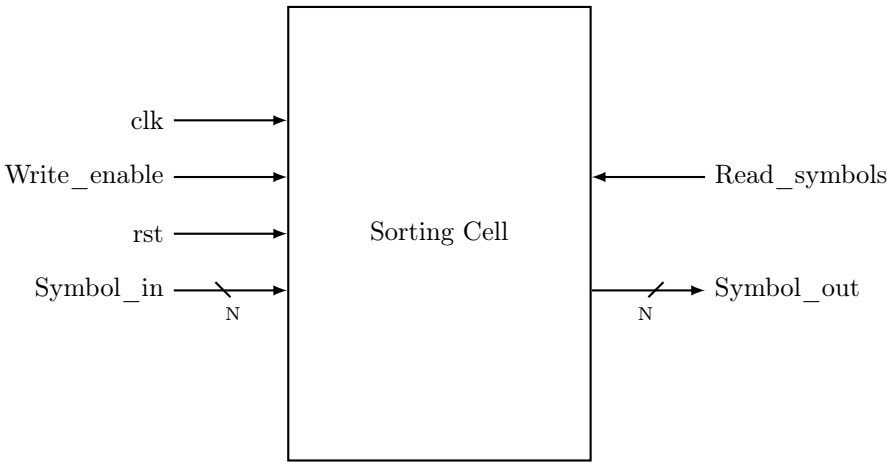

Figura 1: Interfaccia del circuito digitale

L'architettura deve essere **parametrizzabile**, in modo da consentire la configurazione del numero massimo di elementi da ordinare (**M**) e della dimensione dei singoli elementi (**N**).
Il circuito ha il compito di memorizzare gli ultimi M elementi ricevuti in ingresso e ordinarli in **ordine crescente**, considerando gli ingressi di tipo **std_logic_vector, unsigned**.
I simboli in ingresso devono essere letti e processati dal circuito solo se il segnale *Write_enable* risulta essere ad

un valore logico alto. La procedura di lettura viene invece attivata solo se il segnale *Read_ symbols* viene portato a livello logico alto per un ciclo di clock; se ciò accade il circuito fornisce in uscita gli elementi ordinati in ordine crescente.
Il sistema deve gestire correttamente eventuali **situazioni di errore**, quali:

- conflitto di lettura e scrittura simultanea
- overflow del blocco di memoria interno
- richiesta di lettura in assenza di dati disponibili

Viene richiesto inoltre di riportare e discutere i risultati ottenuti nella sintesi/implementazione automatica su FPGA in termini di **massima frequenza di clock** (critical path), **risorse utilizzate** (slice, LUT, etc.) e **potenza dissipata**.

2 Principio di funzionamento

Come mostrato in figura 2, l'architettura è costituita da tre blocchi principali:

- Registro di ingresso **Serial-In Parallel-Out (SIPO)**
- **Sorter**
- Registro di uscita **Parallel-In Serial-Out (PISO)**

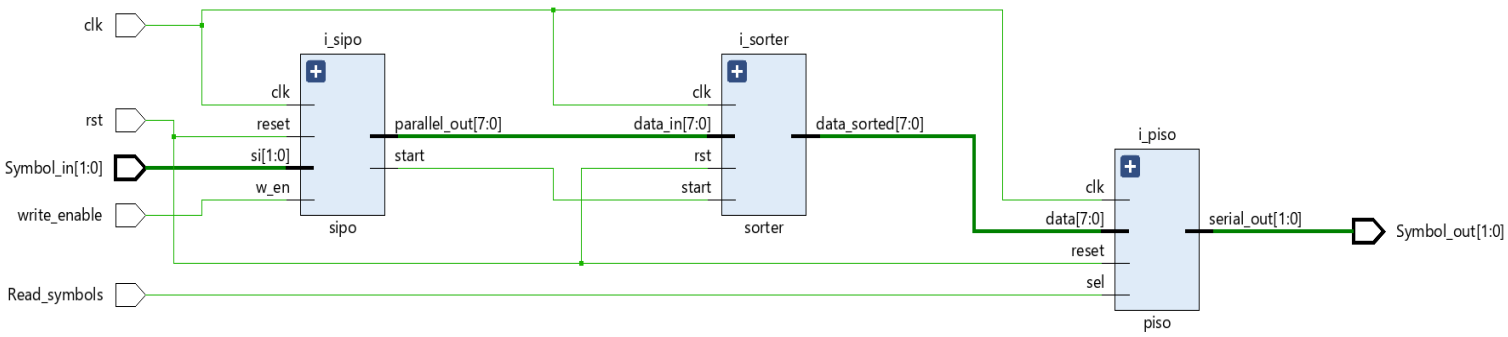

Figura 2: Schema a blocchi della Sorting Cell, generato da Vivado (Elaborated Design)

2.1 SIPO

Il SIPO (figura 3) ha il compito di acquisire i simboli ricevuti in ingresso e predisporli per il successivo ordinamento. Esso riceve al massimo **M simboli** a **N bit ciascuno**. L'acquisizione avviene in modo **seriale**: ad ogni ciclo di clock, se il segnale di controllo *Write_ enable* è alto, viene caricato un nuovo dato. I flip-flop che costituiscono il registro sono infatti dotati di abilitazione (**enable**), così da consentire la scrittura solo quando effettivamente richiesto ed evitare aggiornamenti indesiderati. Dopo M cicli di clock quindi, i simboli risultano essere disponibili in **uscita parallela**, pronti per essere trasferiti al blocco Sorter.
Per garantire la corretta gestione dei simboli, soprattutto nel caso in cui quest' ultimi risultino essere più di M oppure meno, il registro integra anche un **contatore** che tiene traccia di quanti elementi sono stati effettivamente caricati:

- Se il numero di simboli è **inferiore a M**, il registro trasferisce al Sorter solo quelli presenti. Ciò non compromette il funzionamento del Sorter; le posizioni rimaste vuote, a seguito di un reset iniziale, risulteranno inizializzate a zero e il Sorter, come verrà discusso nel paragrafo successivo, durante gli scambi (swap) si troverà semplicemente a muovere questi zeri in alcune posizioni del vettore ordinato.
- Se invece vengono forniti **più di M simboli**, il contatore impedisce ulteriori caricamenti oltre la capacità massima, scartando gli eccessi e forzando l'avvio dell'elaborazione.

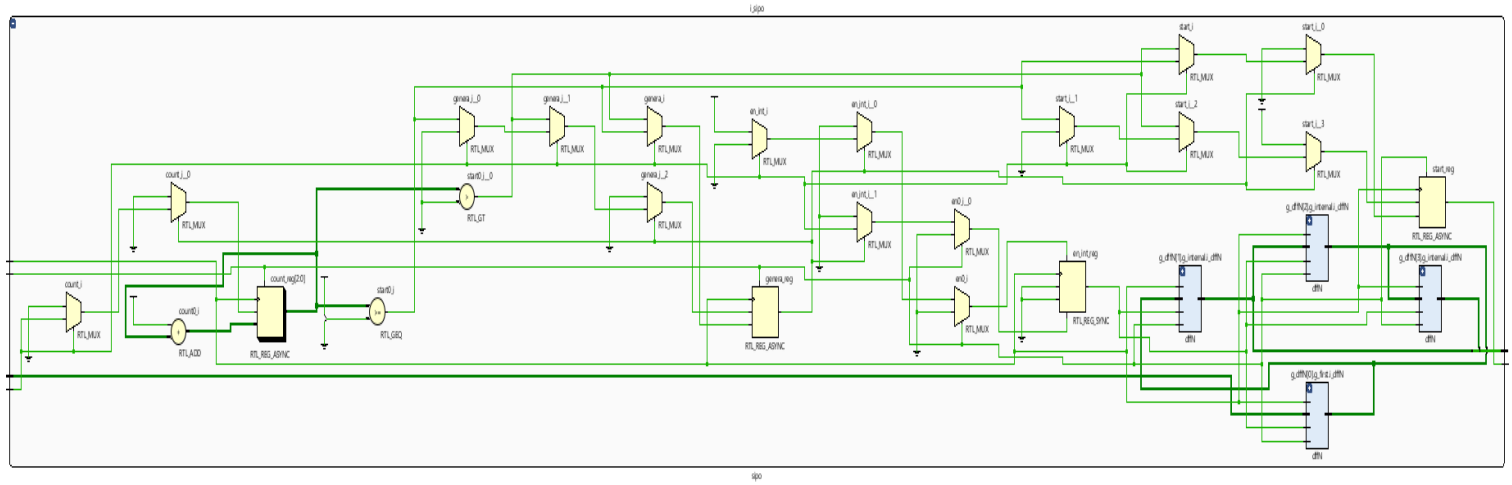

Figura 3: Schematico del SIPO, generato da Vivado (Elaborated Design)

Completato il caricamento dei simboli, il SIPO segnala al blocco Sorter la disponibilità dei dati, asserendo un segnale di sincronizzazione denominato *start* per un ciclo di clock. In questo modo si garantisce il corretto **allineamento** tra i due sottosistemi e si evita che il Sorter lavori su dati incompleti.

2.1.1 Vantaggi del SIPO

Un primo vantaggio dell' introduzione del registro SIPO è la possibilità di **sincronizzare e controllare** con precisione il flusso dei simboli verso il Sorter. Se i dati venissero inviati direttamente al blocco di ordinamento, questo dovrebbe operare in modo continuo e disallineato (iniziando a ordinare mentre i dati non sono ancora tutti disponibili). Con il registro in ingresso invece, il **processo di ordinamento** può essere avviato solo quandi sono disponibili tutti gli elementi; questo permette di avere un **controllo temporale** chiaro sul ciclo di funzionamento e semplifica la logica del Sorter. Il SIPO inoltre, grazie alla presenza del contatore, consente di validare e contare i simboli ricevuti; questa separazione tra la fase di acquisizione e quella di ordinamento rende l'architettura più **robusta e modulare**.

2.1.2 Codice VHDL del blocco SIPO

In figura 4 è mostrata la **entity** del registro di ingresso. La presenza dei due **generici** rendono il blocco **parametrico**, come richiesto; oltre a ciò, troviamo l'ingresso seriale *si*, che permette di caricare i simboli, e l'uscita parallela *parallel_ out* costituita da M simboli ognuno a N bit. Come già anticipato è inoltre presente il segnale *start* che avvisa il Sorter quando i dati sono pronti. I **segnali di controllo** *reset* e *w_ en* servono invece, rispettivamente, per azzerare i flip-flop interni e abilitare la scrittura di nuovi simboli.

```
entity sipo is
generic(
    Nbit : positive := 4;
    M : positive := 4
);
port(
    si : in std_logic_vector(Nbit-1 downto 0);
    parallel_out : out std_logic_vector(M*Nbit-1 downto 0);
    start : out std_logic;
    reset : in std_logic;
    w_en : std_logic;
    clk : in std_logic
);
end entity;
```

Figura 4: Entity del SIPO

In figura 5 viene invece mostrato il **processo** principale, che viene attivato ogni volta che un segnale nella **sensitivity list** varia. Il reset azzera tutti i segnali di interesse; ad ogni fronte del clock, se il segnale *w_en* è alto viene incrementato il **contatore** e attivato *en_int* che permette il caricamento dei dati. Il segnale *genera* è semplicemente un **flag di completamento** che permette di inviare un impulso di *start* una volta terminato il caricamento. Quando il numero di campioni ricevuti raggiunge la capacità massima (**M dati**), oppure se il segnale *w_en* viene abbassato, il processo disattiva l'abilitazione e porta a uno il segnale *start*.

```
        begin
         p_sipo : process(clk, reset)
         begin
             if reset = '1' then
                 count <= 0;
                 start <='0';
                 genera <= '0';
             elsif rising_edge(clk) then
                 if genera = '1' then
                     start <= '0';
                 elsif (w_en = '0' and genera = '0') then ,
                     en_int <= '0';
                     if (count > 0)  then
                         start <= '1';
                         genera <= '1';
                     end if;
                 elsif (w_en = '1' and genera = '0') then
                     en_int <= '1';
                     count <= count + 1;
                     if (count >= M) then
                         start <= '1';
                         genera <= '1';
                     end if;
                 end if;
             end if;
         end process;
```

Figura 5: Processo del SIPO

La restante parte di codice invece, come mostrato in figura 6, serve per **istanziare** M flip-flop (dffN) e per realizzare i **collegamenti** fra di essi; infine vengono assegnati i valori memorizzati all'uscita parallela.

```
      g_dffN : for i in 0 to (M-1) generate
            g_first : if i=0 generate
            i_dffN : dffN
            generic map ( Nbit => Nbit)
            port map(
               clk => clk,
               reset => reset,
               en => en_int,
               di => si,
               do => fifo_mem(i)
            );
            end generate;
            g_internal : if i > 0 and i <= (M-1) generate
            i_dffN : dffN
            generic map ( Nbit => Nbit)
            port map(
                clk => clk,
                reset => reset,
                en => en_int,
                di => fifo_mem(i-1),
                do => fifo_mem(i)
            );
            end generate;
        end generate;

        gen_parallel_out : for i in 0 to (M-1) generate
        parallel_out ((M-i)*Nbit -1 downto (M-i-1)*Nbit) <= fifo_mem(i);
        end generate;
    end architecture;
```

Figura 6: Codice per istanziare i flip-flop e collegare le uscite

2.2 Sorter

Il blocco Sorter (figura 7) costituisce il **cuore della Sorting Cell** e si occupa di ordinare i simboli ricevuti dal registro SIPO; il suo funzionamento è sincrono e viene avviato dall' impulso del segnale di sincronizzazione *start*. Come già anticipato, l'algoritmo adottato è **l'odd-even transposition sort**, che permette una gestione semplice grazie al suo schema regolare.

2.2.1 Odd-Even Transposition Sort

L'algoritmo prevede una sequenza di fasi di confronto e scambio (**compare and swap**), organizzate come segue:

- In una fase **pari**, vengono confrontate le coppie di elementi (0,1), (2,3), (4,5)...e così via.
- In una fase **dispari**, vengono invece confrontate le coppie (1,2), (3,4), (5,6)...
- Ogni confronto avviene in un ciclo di clock: se il simbolo a sinistra nella coppia risulta essere **maggiore** di quello a destra, i due valori vengono scambiati (**swap**).

Un segnale di controllo denominato *phase* tiene conto della fase corrente, così da stabilire se i confronti da effettuare sono quelli pari o quelli dispari.

2.2.2 Latenza

Per garantire l'ordinamento completo di un vettore di M elementi, l'algoritmo utilizzato richiede al massimo M fasi (M cicli di clock); la **latenza** risulta essere quindi, nel caso peggiore, **pari a M**. In molti casi però, il vettore può risultare ordinato anche prima, ma l'architettura **garantisce** il risultato corretto solo dopo M cicli di clock.

2.2.3 Uscita e segnalazione

Al **termine delle fasi** di confronto e scambio, il Sorter pone in uscita il vettore ordinato e **segnala** al registro di uscita (PISO) il completamento dell'operazione, asserendo un segnale di controllo. Questo meccanismo di sincronizzazione consente di avviare correttamente la **serializzazione** in uscita.

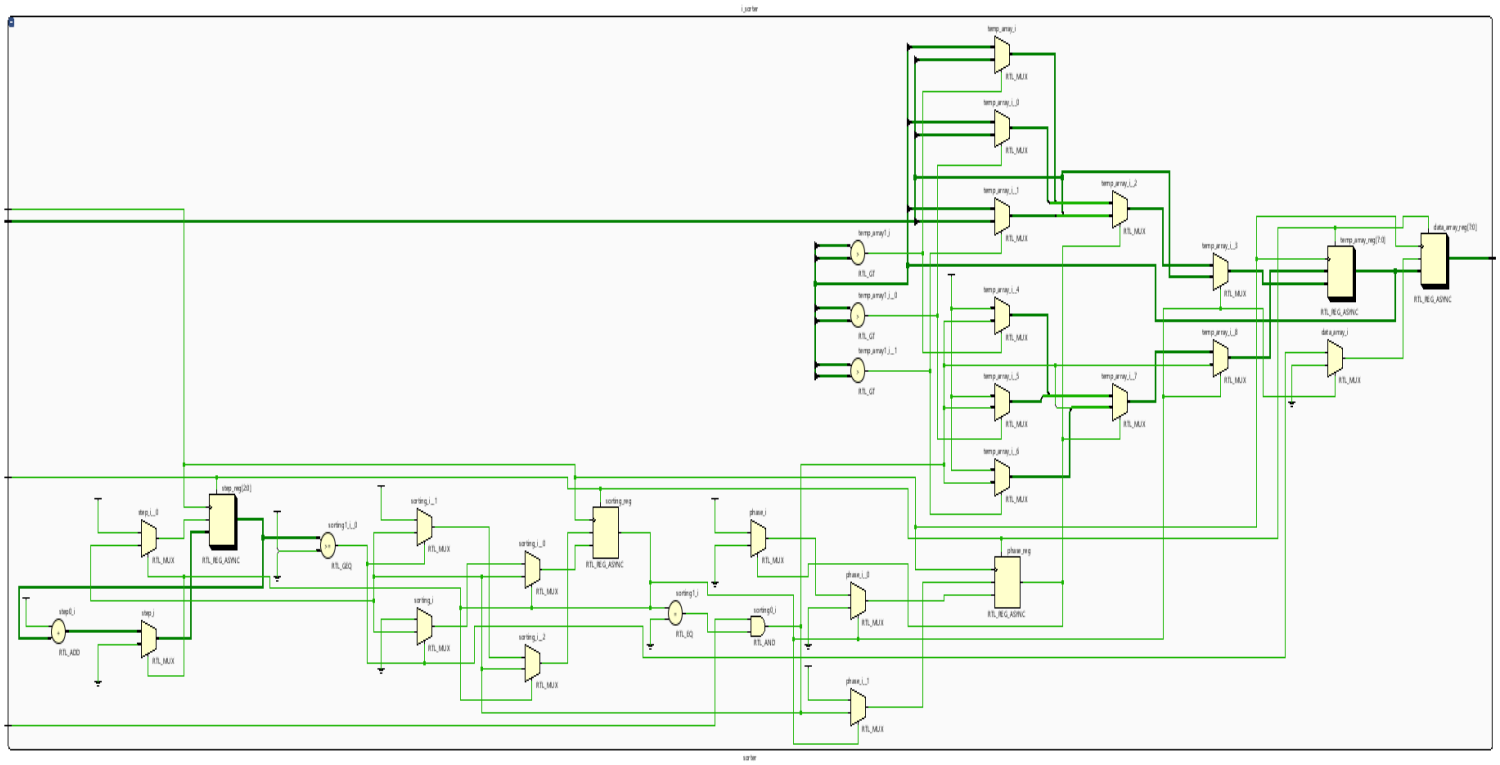

Figura 7: Schematico del Sorter, generato da Vivado (Elaborated Design)

2.2.4 Codice VHDL del blocco Sorter

In figura 8 è mostrata la **entity** del Sorter. Il segnale *start*, collegato all'omonimo segnale in uscita dal SIPO, serve per far partire **l'algoritmo di ordinamento**.

```
  entity sorter is
    generic(
        Nbit : positive := 4;
        M : positive := 4
    );
    port(
        clk : in std_logic;
        rst : in std_logic;
        start : in std_logic;
        data_in : in std_logic_vector(M*Nbit-1 downto 0);
        data_sorted : out std_logic_vector(M*Nbit-1 downto 0)
    );
end entity;
```

Figura 8: Entity del Sorter

In figura 9 viene mostrato il **processo** principale; il Sorter esegue l'algoritmo di ordinamento, come già anticipato, alternando due fasi (**even phase e odd phase**) fino a completare **M step**, garantendo che tutti gli elementi siano **ordinati**.
All'interno del processo viene utilizzata la **variabile** *tmp* come vettore temporaneo per lo scambio di valori tra due elementi dell'array. Nel **caso di swap** infatti:

- Si copia l'elemento corrente in *tmp*.
- Si sovrascrive l'elemento corrente con il successivo.

- Si copia il valore temporaneo *tmp* nel successivo.

I dati sono stati convertiti in **unsigned** poiché il progetto richiedeva l'ordinamento di numeri interi non negativi; in questo modo gli operatori di confronto agiscono correttamente.

```
        begin
    process(clk, rst)
        variable tmp : std_logic_vector(Nbit-1 downto 0);
    begin
        if rst = '1' then
            phase <= '0';
            step <= 0;
            sorting <= '0';
            temp_array <= (others => '0');
            data_array <= (others => '0');
        elsif rising_edge(clk) then
            if start = '1' and sorting = '0' then
                sorting <= '1';
                temp_array <= data_in;
                step <= 0;
                phase <= '0';
            end if;

            if sorting = '1' then
                if phase = '0' then -- Even phase
                    for i in 0 to M-2 loop
                        if i mod 2 = 0 then
                            if unsigned(temp_array((i+1)*Nbit-1 downto i*Nbit)) > unsigned(
                                temp_array((i+2)*Nbit-1 downto (i+1)*Nbit)) then
                                -- Swap
                                tmp := temp_array((i+1)*Nbit-1 downto i*Nbit);
                                temp_array((i+1)*Nbit-1 downto i*Nbit) <= temp_array((i+2)*Nbit-1
                                    downto (i+1)*Nbit);
                                temp_array((i+2)*Nbit-1 downto (i+1)*Nbit) <= tmp;
                            end if;
                        end if;
                    end loop;
                    phase <= '1';
                else -- Odd phase
                    for i in 1 to M-2 loop
                        if i mod 2 = 1 then
                            if unsigned(temp_array((i+1)*Nbit-1 downto i*Nbit)) > unsigned(
                                temp_array((i+2)*Nbit-1 downto (i+1)*Nbit)) then
                                -- Swap
                                tmp := temp_array((i+1)*Nbit-1 downto i*Nbit);
                                temp_array((i+1)*Nbit-1 downto i*Nbit) <= temp_array((i+2)*Nbit-1
                                    downto (i+1)*Nbit);
                                temp_array((i+2)*Nbit-1 downto (i+1)*Nbit) <= tmp;
                            end if;
                        end if;
                    end loop;
                    phase <= '0';
                end if;

                step <= step + 1;
                if step >= M then
                    sorting <= '0';
                    data_array <= temp_array;
                end if;
            end if;
        end if;
    end process;
```

Figura 9: Processo del Sorter

I **vantaggi** di usare una **variabile** rispetto a un segnale sono:

- **Sicurezza dei dati**: l'utilizzo della variabile riduce il rischio di errori di **sovrascrittura**.

- **Aggiornamento immediato**: le variabili in VHDL vengono **aggiornate immediatamente** nel processo, quindi lo scambio avviene istantaneamente dal punto di vista del processo.
- **Maggiore leggibilità e modularità**: senza introdurre segnali aggiuntivi il flusso logico risulta essere più **lineare**, senza necessità quindi di seguire aggiornamenti di segnali distribuiti in più cicli di clock.

L'utilizzo di un segnale al posto della variabile **cambierebbe e complicherebbe** significativamente il funzionamento, in quanto i segnali vengono aggiornati solo alla fine del processo. In tal caso servirebbero **stati e step aggiuntivi**, introducendo anche **ritardi** significativi.
Al termine dell'algoritmo di ordinamento vengono caricati in uscita i dati **ordinati**, pronti per entrare nel registro PISO di uscita.

2.3 PISO

Il registro di uscita è implementato come Parallel-In, Serial-Out (PISO) e svolge la funzione **complementare** rispetto al SIPO: esso riceve il vettore ordinato in parallelo dal blocco Sorter e lo serializza per la trasmissione in uscita.
Ogni flip flop del registro è preceduto, come mostrato in figura 10, da un **multiplexer 2:1**, che seleziona la sorgente dei dati:

- Quando il segnale di selezione *sel* va a livello logico **alto** (collegato al segnale *Read_ symbols* della Sorting Cell), il mux **carica in parallelo** i dati provenienti dal sorter, permettendo al registro di acquisire tutto il vettore ordinato in un singolo ciclo di clock.
- Quando il segnale di selezione *sel* è basso, i flip-flop iniziano a far **scorrere** i dati caricati verso l'uscita, mettendo a disposizione un simbolo alla volta sulla linea seriale *Symbol_ out*.

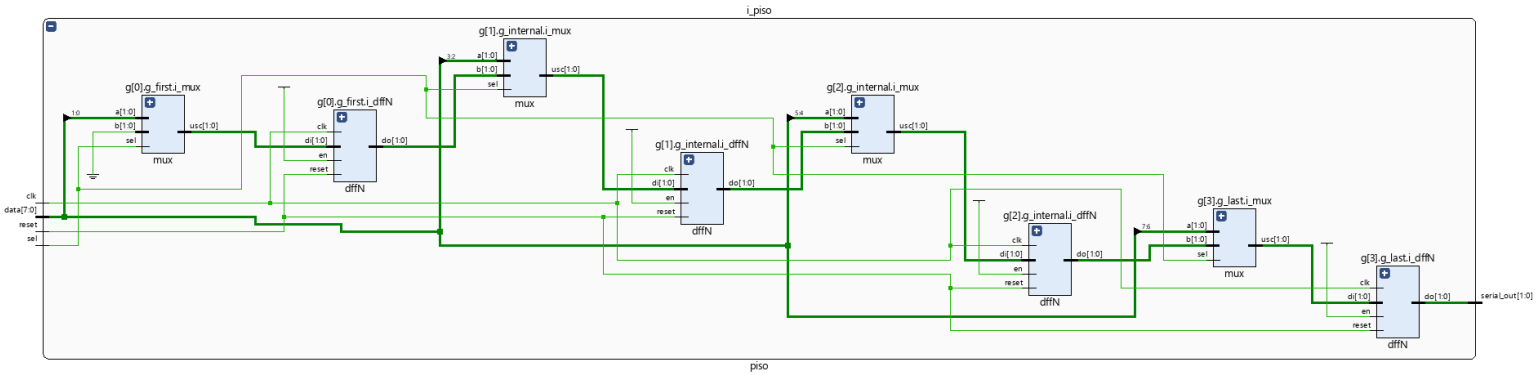

Figura 10: Schematico del PISO, generato da Vivado (Elaborated Design)

Il registro non si limita a serializzare i simboli ordinati, ma introduce un **ulteriore accorgimento**: al termine della sequenza, viene emesso in uscita il valore **0**. Questo valore funge da **flag** per indicare la conclusione dell'array ordinato. Tale scelta progettuale presenta **diversi vantaggi**:

- Consente di distinguere con chiarezza i simboli **validi** dalla fase di inattività;
- Permette ad eventuali moduli successivi di riconoscere automaticamente la **fine della sequenza**, senza la necessità di un conteggio esplicito dei cicli di clock;
- Rende più semplice, come verrà mostrato successivamente, la gestione di input di **lunghezza variabile**.

Tutto ciò è stato ottenuto sfruttando l'ingresso libero del primo multiplexer, posto davanti al primo flip-flop del registro PISO: tale ingresso è stato collegato a una **costante** pari a 0.

2.3.1 Codice VHDL del blocco PISO

In figura 11 è riportata la **entity** del PISO. L'ingresso è il vettore ordinato in uscita dal blocco Sorter, mentre il segnale *sel* serve per controllare i dati in ingresso ai multiplexer 2:1.
La restante parte del codice, in modo analogo a quanto fatto per il blocco SIPO, istanzia i mux e i flip-flop e crea i collegamenti tra di loro, permettendo così la **serializzazione** dei dati in ingresso.

```
 entity piso is
generic(
    Nbit : positive := 4;
    M : positive := 4
);
port(
    data : in std_logic_vector(M*Nbit-1 downto 0);
    serial_out : out std_logic_vector(Nbit-1 downto 0);
    reset : in std_logic;
    clk : in std_logic;
    sel : in std_logic
);
end entity;
```

Figura 11: Entity del PISO

3 Strategia di test

La verifica per il corretto funzionamento della **Sorting Cell** è stata realizzata con un **test-bench** dedicato, progettato per valutarne il comportamento. Utilizzando il simulatore **Modelsim**, sono state riprodotte possibili condizioni operative. All'inizio di ciascun test viene attivato il segnale di **reset**, al fine di portare il sistema in uno stato iniziale noto. Successivamente il reset viene rilasciato e viene condotto il **test** vero e proprio, seguendo la medesima procedura per tutti i casi di verifica. Sia i test-bench che l'analisi successiva effettuata su Vivado sono stati condotti considerando il caso con **M=4** e **Nbit=4**, scelto come configurazione di **riferimento**.

3.1 Primo test: normale funzionamento

Nel primo test (figura 12) è stato verificato il **normale funzionamento** della Sorting Cell. Sono stati forniti in ingresso quattro simboli, ciascuno codificato su quattro bit, e si è osservata l'uscita **ordinata** in modo crescente. Per garantire che l'ordinamento fosse completato, l'impulso di *Read_ symbols* per leggere i dati in uscita è stato generato dopo M+1 cicli di clock dal termine del caricamento dei dati in ingresso.

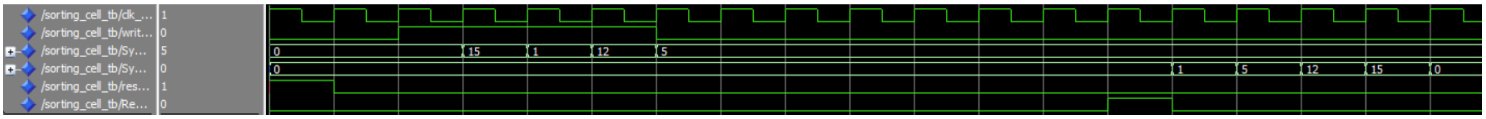

Figura 12: Primo test, funzionamento base

Come previsto dal progetto, una volta visualizzati i simboli ordinati in uscita, la sequenza **termina** con il valore **0**, che agisce da **flag** per segnalare la conclusione dell'array. Questo comportamento è stato correttamente osservato nelle simulazioni. Come si nota, l'uscita assume il valore 0 prima dell'arrivo dei dati ordinati; di conseguenza, per distinguere correttamente un eventuale zero ordinato dal valore di inattività, l'uscita deve essere considerata valida soltanto dopo l'impulso di *Read_ symbols*.

3.2 Secondo test: block overflow

Nel secondo test (figura 13) sono stati inseriti in ingresso **più di M simboli**, al fine di verificare il comportamento del sistema in caso di **block overflow**. Come previsto, i simboli **eccedenti** la capacità della Sorting Cell vengono **ignorati**, mentre in uscita compaiono esclusivamente i primi simboli ordinati. La sequenza termina, come nel caso precedente, con il valore **0**, che funge da **flag** di fine array. Questa scelta progettuale si rivela particolarmente utile

proprio in condizioni di **overflow**, in quanto consente di distinguere chiaramente i dati **validi** da quelli scartati e di segnalare ai moduli successivi la fine della sequenza ordinata, **senza ambiguità**.

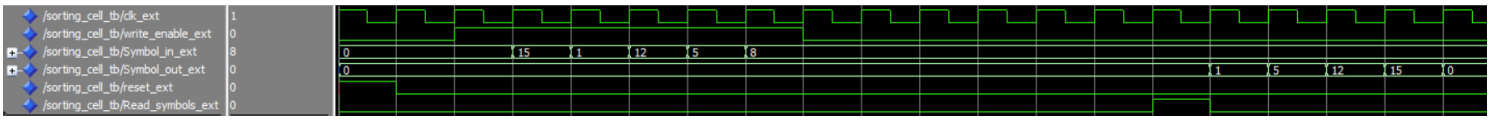

Figura 13: Secondo test, block overflow

3.3 Terzo test: inserimento di meno di M simboli

Nel terzo test (figura 14) sono stati inseriti in ingresso **meno di M simboli**, al fine di verificare il comportamento del sistema in **assenza** del numero massimo di dati previsti. In questo scenario, la Sorting Cell restituisce in uscita i simboli effettivamente caricati, ordinati correttamente, mentre per ogni **posizione mancante** rispetto al numero massimo viene considerato il valore **0**; di conseguenza, poiché l'ordinamento è crescente, tali valori si trovano in testa alla sequenza ordinata. Come nei test precedenti, la sequenza termina con il flag 0.

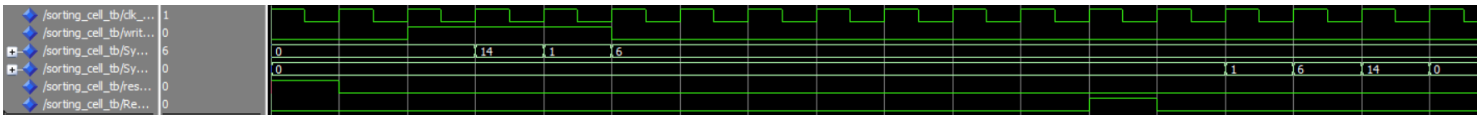

Figura 14: Terzo test, inserimento di meno di M simboli

3.4 Quarto test: conflitto lettura/scrittura

Nel quarto test (figura 15) è stato simulato un caso di **conflitto tra lettura e scrittura**: il segnale *Read_ symbols* è stato attivato dall'esterno mentre il blocco era ancora in fase di scrittura, ovvero con *Write_ enable* alto. Il risultato osservato conferma il comportamento previsto: **durante la scrittura**, ogni tentativo di **lettura** restituisce il valore **0**. Questo meccanismo evita la lettura di **dati parziali** o non ancora **stabilizzati**, garantendo l'integrità dei dati letti.

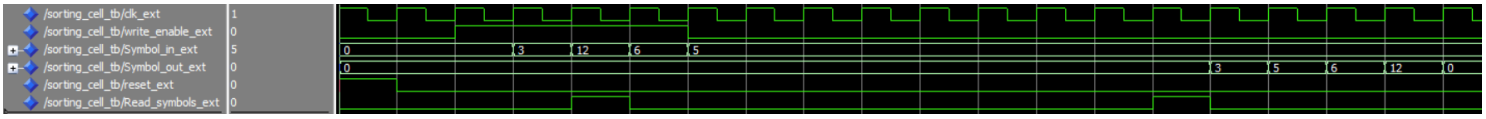

Figura 15: Quarto test, conflitto lettura/scrittura

3.5 Quinto test: lettura in assenza di dati

Nel quinto test (figura 16) è stata verificata la **lettura** in **assenza di dati** inseriti: con nessun simbolo caricato in ingresso, l'uscita **restituisce 0**, confermando che il sistema gestisce correttamente il caso di **lettura a vuoto**. In aggiunta, la figura mostra anche un caso in cui un numero viene **ripetuto** due volte in ingresso (nella simulazione il **valore 12**). L'uscita riflette correttamente questa ripetizione, mantenendo il valore 12 per due cicli di clock consecutivi, prima di passare al simbolo successivo. Questo comportamento conferma che il modulo **supporta** correttamente **simboli duplicati** senza perdita di dati.

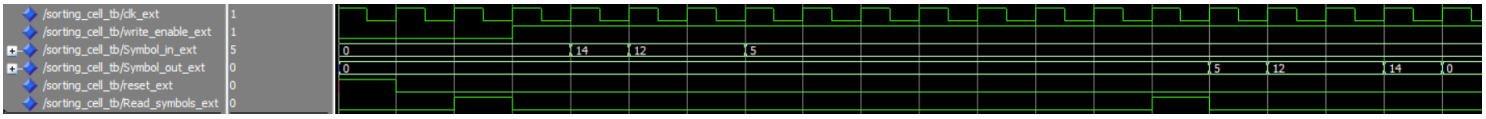

Figura 16: Quinto test, lettura senza dati e gestione dei simboli ripetuti

3.6 Sesto test: altri possibili scenari

L'ultimo test (figura 17) verifica **diversi scenari particolari** relativi al corretto funzionamento della Sorting Cell:

- È stata simulata una scrittura con il segnale *reset* attivo, nonostante *Write_ enable* fosse alto. Come previsto, il simbolo non viene memorizzato e viene **ignorato**.
- Successivamente, è stato tentato un inserimento con *reset* basso, ma con *Write_ enable basso*: anche in questo caso il simbolo **non viene considerato**, confermando che il modulo rispetta i segnali di controllo.
- A questo punto sono stati inseriti correttamente i quattro simboli previsti, verificando la normale sequenza di scrittura.
- È stata effettuata una **lettura anticipata** alzando *Read_ symbols* prima che il sistema avesse **completato** l'ordinamento: in uscita si legge 0, come previsto, evitando la lettura di dati non ancora **stabilizzati**.
- Infine, è stata eseguita una lettura corretta mantenendo però *Read_ symbols* alto per due cicli di clock (il funzionamento normale richiederebbe un solo ciclo). In uscita si osserva che la lettura dei **dati validi** avviene solo dopo che *Read_ symbols* ritorna basso; fino a quel momento, l'uscita **ripete** semplicemente il **primo simbolo** più piccolo.

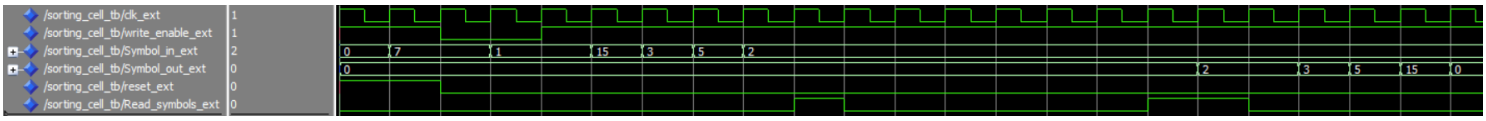

Figura 17: Sesto test, comportamenti combinati e casi particolari

Questo test combina più **situazioni limite** e conferma che il modulo le gestisce correttamente.

4 Sintesi e implementazione su Vivado

La sintesi e l'implementazione della **Sorting Cell** sono state eseguite su **Vivado**, al fine di analizzare le prestazioni del circuito e verificarne l'ottimizzazione su una **FPGA**. Durante la fase di sintesi, sono stati valutati i principali **parametri di progetto**, tra cui la **massima frequenza** di funzionamento, l'utilizzo delle **risorse hardware** (LUT, FF, DSP) e il **consumo energetico**. Per l'implementazione del progetto in Vivado è stata utilizzata come board di riferimento la **Zybo** di Digilent, equipaggiata con un **SoC Xilinx Zynq-7000** (XC7Z010-1CLG400C).

4.1 Wrapper

Vivado valuta il **timing** esclusivamente per i percorsi "**Registro-Logica-Registro**", quindi, per garantire un'analisi accurata, è necessario inserire barriere di registri su tutti gli **ingressi** e le **uscite** del circuito.
Nel progetto in esame, tuttavia, tale accorgimento **non è necessario**. Infatti, l'architettura implementata è già naturalmente delimitata da registri: all'ingresso i dati sono caricati tramite il registro SIPO (**Serial-In Parallel-Out**), mentre all'uscita sono prelevati mediante il registro PISO (**Parallel-In Serial-Out**). La presenza di queste strutture garantisce che i cammini considerati da Vivado siano già del tipo "Registro-Logica–Registro", permettendo quindi di svolgere l'analisi di timing **senza** l'implementazione di alcun **wrapper**.

4.2 Sintesi logica

La sintesi logica in Vivado è il processo mediante il quale il **codice RTL** viene tradotto in una rappresentazione gate-level del circuito e, durante questa fase, viene analizzata la descrizione del design, ottimizzando l'utilizzo delle **risorse hardware** e verificando i **vincoli di timing** per garantire il corretto funzionamento della Sorting Cell.
Utilizzando una frequenza di clock pari a $f_{clk} = 125\,\text{MHz}$, in figura 18 viene mostrato il **Report Time Summary** generato da Vivado.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	4,260 ns	Worst Hold Slack (WHS):	0,254 ns	Worst Pulse Width Slack (WPWS):	3,500 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	123	Total Number of Endpoints:	123	Total Number of Endpoints:	75

All user specified timing constraints are met.

Figura 18: Design Timing Summary, sintesi logica

Il **Worst Negative Slack** (WNS) risulta essere **positivo**, indicando che il design soddisfa il **vincolo di timing** e potrebbe operare a **frequenze superiori**. In particolare si ottiene un WNS di 4.260 ns e, a partire da questo, è possibile determinare la **massima** frequenza di clock **teorica**, utilizzando la seguente formula:

$$f_{max} = \frac{1}{(T_{clk} - WNS)} = 267.379MHz \tag{1}$$

Questa frequenza è limitata dal **Critical Path**, mostrato in figura 19; esso è localizzato interamente all'interno del Sorter e coinvolge le operazioni di confronto e scambio degli elementi, che vengono gestite attraverso il vettore temporaneo *temp_ array*.

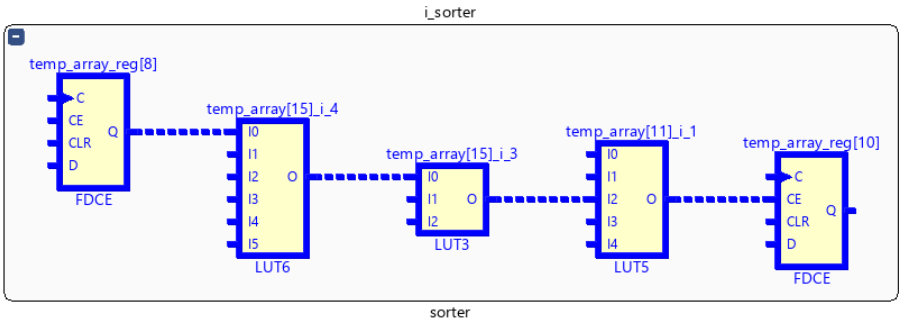

Figura 19: Critical Path, sintesi logica

4.3 Implementazione

L'implementazione su Vivado rappresenta la **fase finale** del processo di progettazione di un circuito digitale, nella quale il design descritto in VHDL viene tradotto in una configurazione concreta per FPGA.
Durante questa fase vengono allocati gli elementi logici sintetizzati nella fase precedente, ottimizzando i collegamenti tra un elemento e l'altro e viene fornita una nuova verifica finale dei vincoli temporali, includendo anche il tempo di propagazione dei segnali nei percorsi logici.
A seguito dell'implementazione della Sorting Cell è stato generato il **Report Time Summary** mostrato in figura 20, dove si può osservare che il **Worst Negative Slack** (WNS) risulta essere **ancora positivo**, confermando che il design continua a soddisfare il vincolo di timing, ma si nota anche che questo è leggermente **diminuito** rispetto alla fase precedente, proprio perché ora si tiene conto anche dei **ritardi** introdotti dalle **interconnessioni**.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	4,113 ns	Worst Hold Slack (WHS):	0,113 ns	Worst Pulse Width Slack (WPWS):	3,500 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	123	Total Number of Endpoints:	123	Total Number of Endpoints:	75

All user specified timing constraints are met.

Figura 20: Design Timing Summary, implementazione

A partire dal WNS è possibile calcolare adesso la massima frequenza di clock **reale**, utilizzando la formula precedentemente citata:

$$f_{max} = \frac{1}{(T_{clk} - WNS)} = 257,26MHz \tag{2}$$

Essa è, ovviamente, inferiore a quella teorica, ma il vincolo temporale impostato permette comunque di rientrare ampiamente nella massima frequenza raggiungibile. L'analisi del **Critical Path** nell'implementazione evidenzia sempre lo stesso percorso, mostrato in figura 19.

4.3.1 Risorse utilizzate

L'analisi delle **risorse utilizzate** (figura 21) nella Sorting Cell implementata mostra che il design impiega **47 Slice LUT**, **75 Slice Registers** e **19 Slice**. L'occupazione complessiva risulta essere quindi molto **contenuta** rispetto alle risorse disponibili sul dispositivo.
Come ci si attendeva, il **Sorter** risulta il componente più **oneroso** sia in termini di logica combinatoria sia di registri. I moduli **PISO** e **SIPO**, invece, mostrano un numero relativamente elevato di registri in proporzione alla logica utilizzata, caratteristica tipica delle strutture sequenziali dedicate al trasferimento e all'allineamento dei dati. Nel complesso, l'**architettura** dimostra di essere **leggera** e facilmente scalabile, lasciando ampio margine per l'integrazione di più celle di ordinamento in un sistema più complesso.

Name	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)
sorting_cell	47	75	19	47
i_piso (piso)	8	16	2	8
i_sipo (sipo)	4	22	6	4
i_sorter (sorter)	35	37	14	35

Figura 21: Risorse utilizzate

4.3.2 Consumo di potenza

In Vivado la **stima** della **potenza consumata** viene effettuata attraverso strumenti di analisi che considerano sia il consumo statico sia il consumo dinamico.

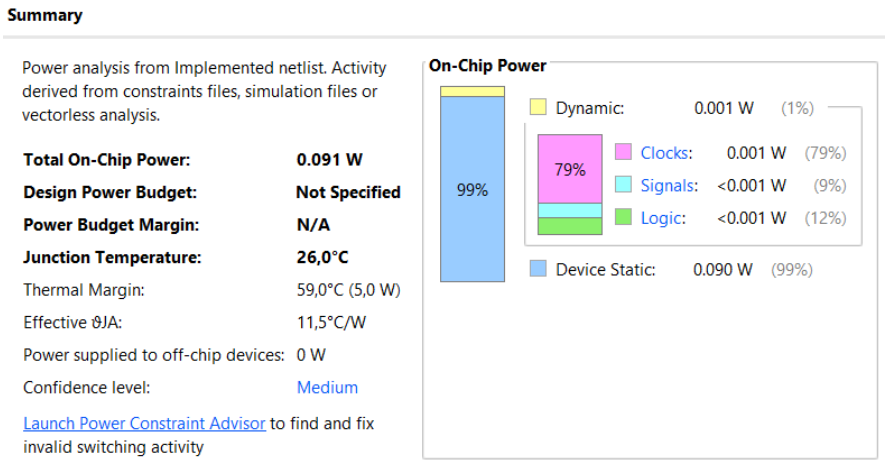

Figura 22: Stima della potenza dissipata

Nel caso della Sorting Cell implementata, il **Report Power Summary** (mostrato in figura 22) ha evidenziato una **potenza totale** dissipata pari a **0.091 W**, suddivisa in **0.090 W** di potenza **statica** e **0.001 W** di potenza **dinamica**. Questo risultato è coerente con le caratteristiche del progetto: la Sorting Cell presenta una logica molto ridotta, con poche LUT e registri, e non fa uso di blocchi ad alta intensità di commutazioni (DSP, BRAM o reti di clock complesse). Inoltre, la frequenza di lavoro è relativamente bassa e il numero di transizioni logiche è limitato, portando la potenza dinamica a valori quasi nulli. La **quasi totalità** del consumo è quindi imputabile alla **potenza statica**, presente anche in **assenza** di attività di commutazione.

4.3.3 Warning

Nella fase di implementazione sono stati generati **11 warning** (figura 23), principalmente relativi alla gestione delle proprietà HD.CLK_SRC e HD.PARTPIN_LOCS in modalità **out-of-context**.
Questi avvisi segnalano che Vivado non ha potuto determinare automaticamente la **sorgente** del clock e la posizione dei **pin** per alcuni segnali, compromettendo l'accuratezza dell'analisi del timing.

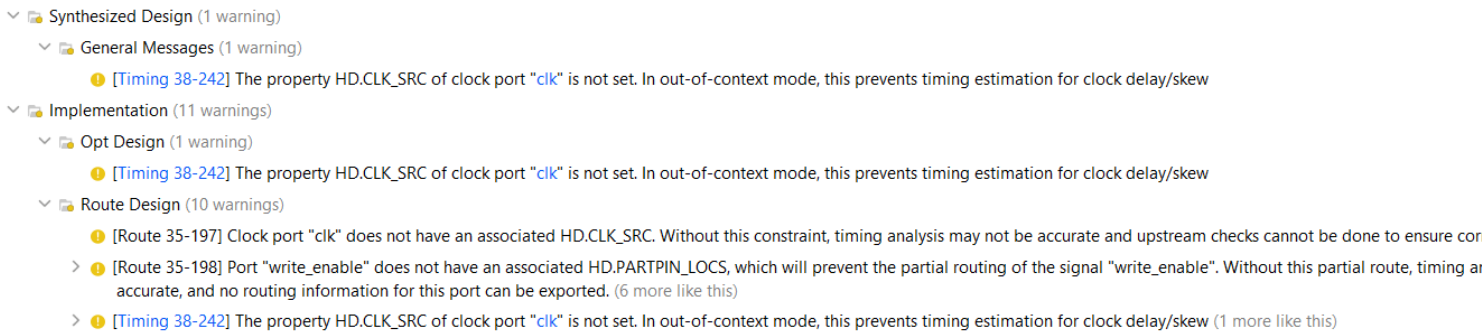

Figura 23: Warning

Inoltre è stato generato un **warning** riguardante il mancato utilizzo della **cella PS7** (figura 24), che dovrà essere utilizzata per poter garantire una configurazione corretta del design su piattaforma **Zynq**.

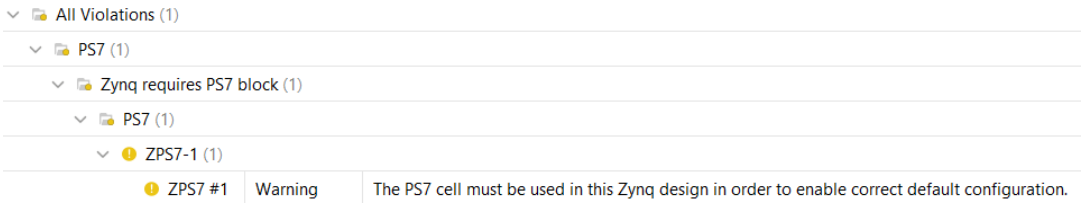

Figura 24: Warning, PS7

5 Conclusioni

Il progetto assegnato prevedeva l'implementazione della **Sorting Cell** su **FPGA**, utilizzando **VHDL** per la descrizione hardware, **Modelsim** per la simulazione funzionale e **Vivado** per la sintesi e l'implementazione.
Nonostante non sia stata assegnata una **costraint** per la frequenza di clock, è stato scelto di utilizzare $f_{clk} = 125\,\text{MHz}$ e l'architettura progettata centra questa specifica, oltre ad implementare la Sorting Cell tenendo conto delle specifiche di progetto. L'implementazione scelta, nel caso specifico di 4 simboli a 4 bit ciascuno, ha evidenziato un'occupazione molto ridotta delle risorse logiche e un consumo di potenza estremamente contenuto, dominato quasi esclusivamente dalla componente statica del dispositivo.
Dal punto di vista funzionale, la cella **rispetta** i **requisiti richiesti**, consentendo di ricevere i dati in ingresso, ordinarli e fornirli correttamente in uscita. È stato scelto di forzare l'uscita a **zero** dopo la fine del vettore ordinato, in modo da avere un segnale chiaro che indichi la conclusione dei dati validi.
Nel progetto sviluppato si è scelto deliberatamente di mantenere l'architettura **semplice**, aderendo strettamente alla specifica fornita. Per questo motivo è stato deciso di utilizzare soltanto i segnali previsti nell'**entity**, senza introdurre ulteriori **flag** o **protocolli** di comunicazione. Tale scelta ha reso necessario richiedere all'utente di resettare la macchina prima di ogni nuovo caricamento di simboli: in questo modo si garantisce che i registri interni vengano azzerati e che il sistema riparta sempre da uno stato noto, evitando comportamenti indesiderati. Questa soluzione, pur funzionando correttamente, ha alcune **limitazioni** dal punto di vista pratico. In un sistema reale, infatti, sarebbe preferibile permettere alla cella di autogestirsi in modo più robusto. A tale scopo, diverse **migliorie** possono essere introdotte:

- **Flag di validità dei dati in uscita**: un segnale *data_valid* o *end_of_sequence* consentirebbe di distinguere chiaramente quando i simboli forniti dal PISO sono validi e quando l'uscita non contiene informazioni

significative. In questo modo non sarebbe più necessario utilizzare valori sentinella (come lo zero aggiunto al termine della sequenza).

- **Segnale di "busy" o "ready"**: un flag di stato potrebbe indicare quando la Sorting Cell è occupata nell'elaborazione o quando è pronta a ricevere nuovi dati.
- **Gestione automatica del reset interno**: la cella potrebbe integrare un meccanismo di reset parziale interno che si attiva automaticamente una volta completato l'ordinamento e la lettura dei simboli. In questo modo il blocco tornerebbe in uno stato iniziale pronto a ricevere nuovi dati senza interventi esterni.
- **Protocolli di handshaking**: l'introduzione di protocolli standard come *ready/valid* permetterebbe una comunicazione più affidabile tra moduli diversi. Con tale approccio, ogni trasferimento avverrebbe solo quando entrambe le parti sono pronte, garantendo robustezza e flessibilità.

In sintesi, il progetto attuale rappresenta una soluzione funzionale e coerente con la specifica fornita, ma in contesti più complessi si renderebbe necessario un potenziamento dell'interfaccia di controllo per migliorare l'autonomia, l'affidabilità e l'integrazione con altri moduli.
Un'altra prospettiva di miglioramento riguarda l'**algoritmo di ordinamento** impiegato. Nel progetto attuale, la logica è stata progettata in maniera semplice e lineare, senza ricorrere a strutture algoritmiche particolarmente complesse. Tuttavia, esistono **diverse strategie alternative**:

- **Bubble sort**: semplice da implementare in hardware, ma poco efficiente in termini di velocità per insiemi numerosi di dati, poiché richiede molte iterazioni.
- **Insertion sort**: ancora semplice, ma più efficiente del Bubble per array piccoli; mantiene comunque una complessità quadratica.
- **Merge sort** o **Quick sort**: algoritmi molto più efficienti dal punto di vista della complessità temporale (O(n log n)), ma con un'implementazione hardware più onerosa in termini di risorse logiche e di gestione della memoria.
- **Bitonic sort**: particolarmente adatto per architetture hardware parallele; richiede più risorse, ma garantisce tempi di ordinamento costanti e ben deterministici.

La **scelta** dell'algoritmo, in un contesto reale, dipenderebbe quindi dal **compromesso** desiderato tra **velocità di esecuzione**, **area occupata** e **potenza consumata**. Nel caso di FPGA, infatti, esiste spesso un bilanciamento da ottimizzare: architetture più parallele consentono latenze minori, ma richiedono un numero molto più elevato di LUT e registri.
Infine, la **natura parametrica** della cella apre la strada a una facile estensione verso array di dimensioni maggiori o simboli a più bit, senza modificare l'architettura di base. In sistemi complessi, più celle potrebbero essere interconnesse in catene o reti per costruire ordinatori più potenti (ad esempio **sorting networks**), rendendo il progetto scalabile e riutilizzabile in diversi contesti applicativi.
In sintesi, il **lavoro svolto** ha fornito un'implementazione leggera e facilmente estendibile di una cella di ordinamento. Al tempo stesso, ha messo in evidenza le possibili direzioni di miglioramento e le alternative algoritmiche che potrebbero essere adottate per ottimizzare le prestazioni in funzione dei vincoli progettuali di area, potenza ed efficienza.